Lecture: Retrieval-Augmented Generation

Bridging Knowledge Retrieval and Language Generation

Advanced Natural Language Processing

Overview

Large Language Models (LLMs) are remarkable, but they have a fundamental problem: they only know what they learned during training. Ask about yesterday's news, your company's internal documents, or any specialized domain knowledge, and you'll likely get either outdated information or confident-sounding hallucinations. Retrieval-Augmented Generation (RAG) elegantly solves this by giving LLMs the ability to "look things up" before answering. Today we'll build a complete understanding of RAG—from the core intuition to the mathematical foundations to practical implementation considerations.

1 The Problem: Why LLMs Need External Knowledge

1.1 Limitations of Parametric Knowledge

LLMs store knowledge in their parameters—billions of weights learned from training data. This **parametric knowledge** has serious limitations:

- **Knowledge Cutoff**: Training data has a fixed endpoint. An LLM trained in 2023 knows nothing about events in 2024.
- **Hallucinations**: When LLMs lack knowledge, they often generate plausible-sounding but false information with high confidence.
- **No Access to Private Data**: LLMs cannot access your company's internal documents, databases, or any proprietary information.
- **Costly Updates**: Retraining an LLM to incorporate new knowledge requires enormous computational resources.

1.2 The RAG Solution

RAG introduces **non-parametric memory**—an external knowledge base that the model can query at inference time. Instead of answering purely from what it "remembers," the model first retrieves relevant information, then generates a response grounded in that retrieved context.

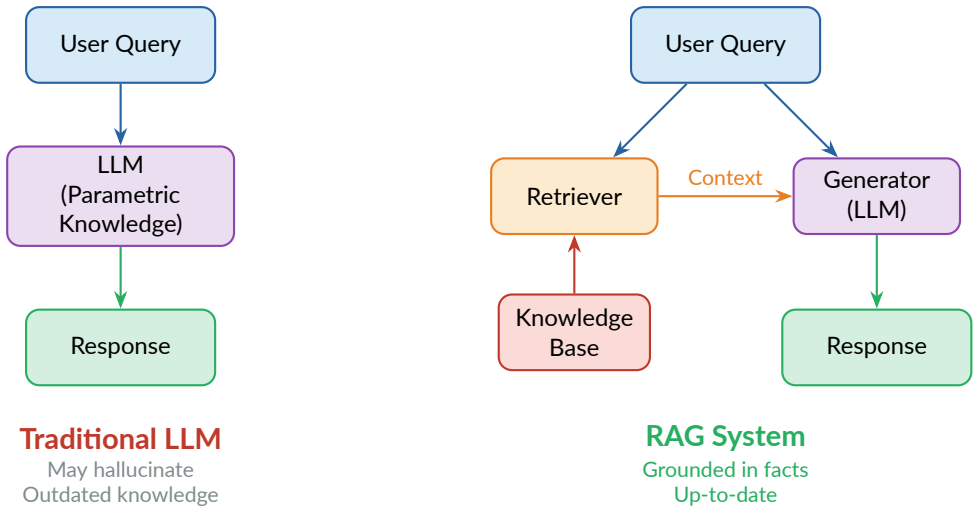


Figure 1: Traditional LLM vs. RAG architecture. RAG retrieves relevant context from an external knowledge base before generating a response.

2 The RAG Architecture

RAG systems have three core components that form a pipeline: **Indexing**, **Retrieval**, and **Generation**. Let's examine each in detail.

2.1 Component 1: The Knowledge Base (Indexing)

Before retrieval can happen, we need to prepare our documents for efficient search. This involves several steps:

1. **Loading**: Import documents from various sources (PDFs, databases, web pages, APIs).
2. **Chunking**: Break large documents into smaller pieces called *chunks*. Typical chunk sizes range from 256 to 1024 tokens. The chunking strategy significantly impacts retrieval quality.
3. **Embedding**: Convert each chunk into a dense vector representation using an embedding model. This vector captures the semantic meaning of the text.
4. **Storing**: Save these vectors in a *vector database* optimized for similarity search (e.g., Pinecone, Weaviate, Chroma, FAISS).

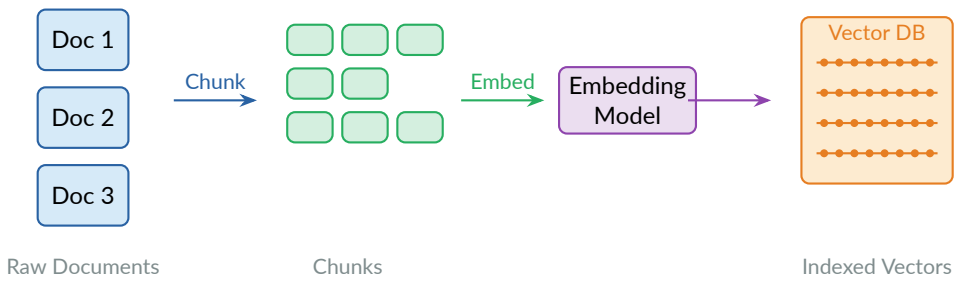


Figure 2: The indexing pipeline: documents are chunked, embedded into vectors, and stored in a vector database for efficient similarity search.

query যদি বড় হয় → chunk এ divide → embedding generation এ দিবে

Query র embedding and knowledgebase embedding এর মাঝে similarity score calc করবো।
Query র সাথে related data retrieve করবো।
e.g. cosine, dot product etc.

2.2 Component 2: The Retriever

When a user query arrives, the retriever finds the most relevant chunks from the knowledge base. The query goes through the *same embedding model* used for indexing, producing a query vector. We then search for document vectors that are "closest" to this query vector.

Definition 2.1 (Retriever). A retriever is a function $R : \mathcal{Q} \to \mathcal{P}(\mathcal{D})$ that maps a query q to a subset of documents (or chunks) from the corpus $\mathcal{D}$, typically returning the top-k most relevant items.

Modern retrievers use **dense retrieval**—representing both queries and documents as dense vectors learned by neural networks, rather than sparse keyword-based methods like TF-IDF or BM25.

user prompt modify করে then পাঠানো হয় model এ.

2.3 Component 3: The Generator

preprocess → যেই format এ llm learn করেছে সেই format এ নেয়া হয়।

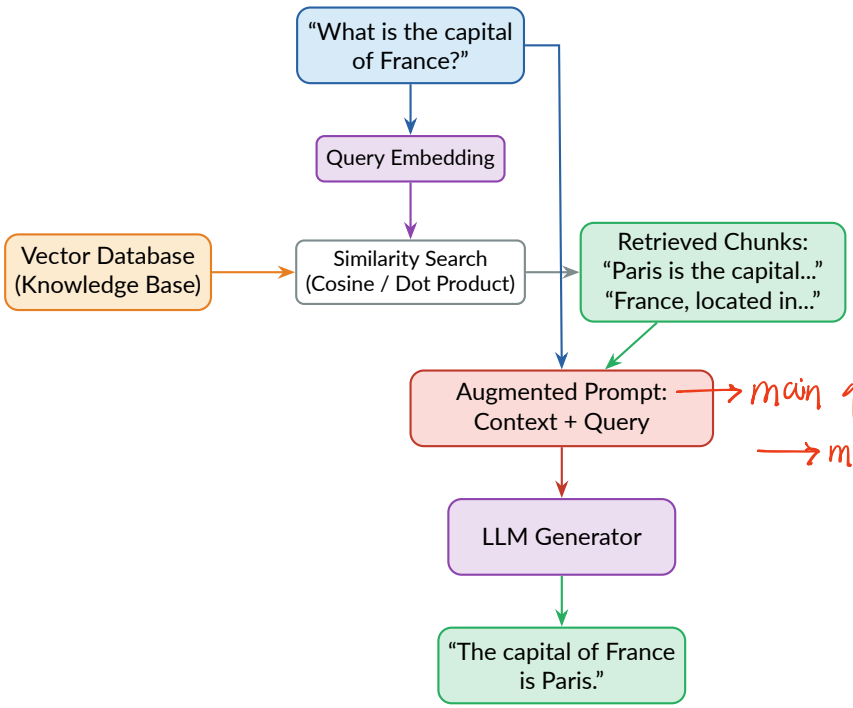
The generator (typically an LLM) receives the original query *augmented* with the retrieved context and produces the final response. The augmented prompt typically looks like:

```
Context: [Retrieved chunk 1] [Retrieved chunk 2] ...

Question: [User's original query]

Answer based on the context above:
```

This grounds the LLM's response in actual retrieved facts rather than relying solely on parametric knowledge.



main query + context → modified prompt

Figure 3: Complete RAG pipeline: the query is embedded, similar chunks are retrieved via vector similarity search, and the LLM generates a response grounded in the retrieved context.

কখন RAG based setup useful?

3 Mathematical Foundations

3.1 Embeddings: From Text to Vectors

An **embedding model** is a function $E : \mathcal{T} \to \mathbb{R}^d$ that maps text (words, sentences, or paragraphs) to d-dimensional vectors. These vectors are designed so that semantically similar texts have similar vector representations.

Modern embedding models (like Sentence-BERT, OpenAI's text-embedding-ada-002, or Cohere's embed) are trained using contrastive learning objectives to cluster similar meanings together in the embedding space.

Example 3.1. Consider embeddings for the following texts:

- "The cat sat on the mat" $\to \mathbf{v}_1$
- "A feline rested on the rug" $\to \mathbf{v}_2$
- "Stock prices rose yesterday" $\to \mathbf{v}_3$

Despite different words, $\mathbf{v}_1$ and $\mathbf{v}_2$ will be close in embedding space (high similarity), while $\mathbf{v}_3$ will be distant from both (low similarity).

3.2 Similarity Metrics

Once we have vector representations, we need a way to measure "closeness." The most common metrics are:

3.2.1 Cosine Similarity

Cosine similarity measures the cosine of the angle between two vectors:

$$\text{sim}_{\cos}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \frac{\sum_{i=1}^{d} a_i b_i}{\sqrt{\sum_{i=1}^{d} a_i^2} \cdot \sqrt{\sum_{i=1}^{d} b_i^2}}$$

where $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ are embedding vectors. The result ranges from -1 to 1:

- $+1$: Vectors point in exactly the same direction (semantically identical)
- 0: Vectors are orthogonal (no semantic relationship)
- -1: Vectors point in opposite directions (semantically opposite)

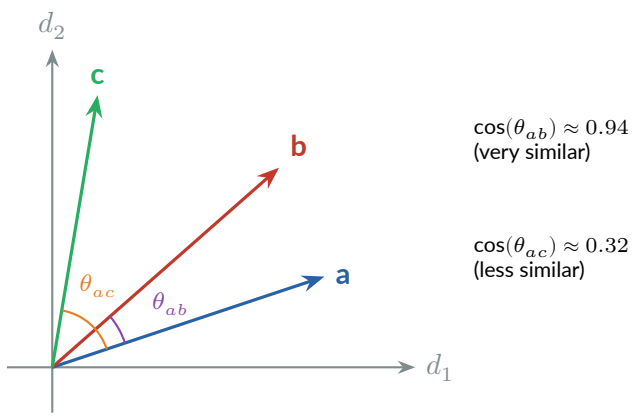


Figure 4: Cosine similarity measures the angle between vectors. Smaller angles (higher cosine) indicate greater semantic similarity.

3.2.2 Dot Product

For **normalized vectors** (unit length, $\|\mathbf{a}\| = \|\mathbf{b}\| = 1$), the dot product equals cosine similarity:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{d} a_i b_i$$

Most modern embedding models output normalized vectors, so dot product is often preferred for computational efficiency (no division needed). Higher dot products indicate greater similarity.

3.2.3 Euclidean Distance

Euclidean (L2) distance measures the straight-line distance between vector endpoints:

$$d_{\text{euclidean}}(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{i=1}^{d} (a_i - b_i)^2}$$

Unlike similarity metrics, *smaller* Euclidean distances indicate *greater* similarity.

3.3 The Retrieval Process

Given a query q and a corpus of N document chunks $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$:

1. Compute query embedding: $\mathbf{q} = E(q)$
2. For each chunk d_i, compute similarity: $s_i = \text{sim}(\mathbf{q}, \mathbf{d}_i)$
3. Return the top-k chunks with highest similarity scores

Algorithm 1 Basic RAG Retrieval

Require: Query q, Corpus embeddings $\{\mathbf{d}_1, \dots, \mathbf{d}_N\}$, Number of results k
Ensure: Top-k most relevant chunks

```
q ← E(q)                                   ▷ Embed the query
scores ← []
for i = 1 to N do
    s_i ← sim_cos(q, d_i)                  ▷ Compute similarity
    scores.append((i, s_i))
end for
return top_k(scores, k)                    ▷ Return indices of top-k scores
```

3.4 Approximate Nearest Neighbor (ANN) Search

Exact similarity search over millions of vectors is computationally expensive ($O(N)$ comparisons per query). In practice, we use **Approximate Nearest Neighbor (ANN)** algorithms that trade a small amount of accuracy for dramatic speed improvements.

Common ANN approaches include:

- **HNSW** (Hierarchical Navigable Small World): Builds a graph structure for efficient navigation
- **IVF** (Inverted File Index): Clusters vectors and searches only relevant clusters
- **PQ** (Product Quantization): Compresses vectors for memory-efficient search

ANN algorithms can search billions of vectors in milliseconds with $> 95\%$ recall of the true nearest neighbors.

4 Chunking Strategies

How you split documents into chunks dramatically affects retrieval quality. The goal is to create chunks that are small enough to be specific but large enough to retain context.

4.1 Fixed-Size Chunking

The simplest approach: split text every n tokens (e.g., 512 tokens) with optional overlap.

- **Pros**: Simple, fast, predictable memory usage
- **Cons**: May cut sentences mid-thought, separating questions from answers

4.2 Semantic Chunking

Split at natural boundaries—paragraphs first, then sentences, with token limits as fallback.

- **Pros**: Preserves semantic structure, keeps related content together
- **Cons**: Variable chunk sizes, more complex implementation

4.3 Recursive Chunking

Hierarchically split using multiple separators (sections $\rightarrow$ paragraphs $\rightarrow$ sentences) until chunks reach target size.

4.4 Contextual Retrieval (Advanced)

A technique introduced by Anthropic in 2024: before embedding, prepend a short contextual summary to each chunk explaining where it fits in the document. A chunk reading "The revenue was $3.2B" becomes "This chunk is from Acme Corp's Q3 2025 earnings report. The revenue was $3.2B." This provides critical context that would otherwise be lost.

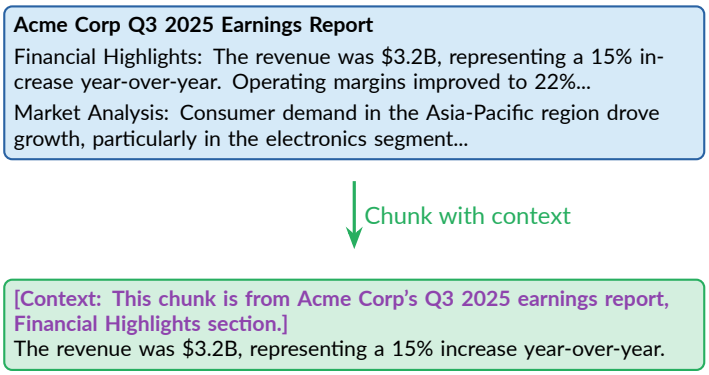


Figure 5: Contextual chunking adds document-level context to each chunk, improving retrieval accuracy.

5 Formal RAG Formulation

Let's formalize RAG mathematically, following the original formulation by Lewis et al. (2020).

5.1 Problem Setup

- **Input**: Query x (e.g., a question)
- **Output**: Response y (e.g., an answer)
- **Knowledge Base**: Collection of documents $\mathcal{D} = \{z_1, z_2, \dots, z_N\}$
- **Retriever**: $p_\eta(z|x)$ — probability of retrieving document z given query x
- **Generator**: $p_\theta(y|x, z)$ — probability of generating y given query x and retrieved context z

5.2 RAG-Sequence Model

In RAG-Sequence, the entire output sequence is generated conditioned on a single retrieved document. The marginal probability is:

$$p_{\text{RAG-Seq}}(y|x) = \sum_{z \in \text{top-}k(\mathcal{D})} p_\eta(z|x) \cdot p_\theta(y|x, z)$$

The model retrieves the top-k documents, generates a response for each, and marginalizes over all retrieved documents.

5.3 RAG-Token Model

In RAG-Token, different tokens can attend to different retrieved documents:

$$p_{\text{RAG-Token}}(y|x) = \prod_{i=1}^{n} \sum_{z \in \text{top-}k(\mathcal{D})} p_\eta(z|x) \cdot p_\theta(y_i|x, z, y_{1:i-1})$$

This allows the model to combine information from multiple documents within a single response.

5.4 Retriever Probability

The retriever uses a bi-encoder architecture with separate encoders for queries and documents:

$$p_\eta(z|x) \propto \exp\left(E_Q(x)^\top E_D(z)\right)$$

where E_Q is the query encoder and E_D is the document encoder. This is essentially a softmax over dot-product similarities.

6 Advanced RAG Techniques

The basic RAG pipeline can be enhanced in numerous ways. Here are the most impactful techniques.

6.1 Query Enhancement

6.1.1 Query Rewriting

Transform the user's query into a form better suited for retrieval. For example, "What happened yesterday?" becomes "News events [current date - 1 day]."

6.1.2 Hypothetical Document Embeddings (HyDE)

Generate a hypothetical answer to the query, then use that answer's embedding for retrieval. This can bridge the semantic gap between question-style queries and declarative document content.

6.1.3 Multi-Query Retrieval

Generate multiple reformulations of the query, retrieve documents for each, and combine results using techniques like Reciprocal Rank Fusion (RRF).

6.2 Retrieval Enhancement

6.2.1 Hybrid Search

Combine dense retrieval (embeddings) with sparse retrieval (BM25/keyword matching). This captures both semantic similarity and exact keyword matches.

6.2.2 Reranking

Use a more sophisticated model (like a cross-encoder) to rerank the initial retrieval results. Cross-encoders are more accurate but slower, so they're applied only to the top candidates.

Figure 6: Two-stage retrieval: a fast bi-encoder retrieves many candidates, then a slower cross-encoder reranks them for higher precision.

6.3 Self-RAG and Adaptive Retrieval

Not every query needs retrieval—sometimes the LLM already knows the answer. **Self-RAG** teaches models to decide *when* to retrieve and to verify their own outputs against retrieved evidence. This reduces unnecessary retrieval calls and improves efficiency.

6.4 GraphRAG

Traditional RAG struggles with questions requiring synthesis across multiple documents or understanding of relationships. **GraphRAG** organizes knowledge as a graph, enabling multi-hop reasoning and relationship traversal.

7 Evaluating RAG Systems

RAG evaluation is challenging because errors can occur at multiple stages. A comprehensive evaluation examines both retrieval quality and generation quality.

7.1 Retrieval Metrics

- **Precision@k**: Fraction of retrieved documents that are relevant
- **Recall@k**: Fraction of all relevant documents that were retrieved
- **MRR (Mean Reciprocal Rank)**: Average of $1/\text{rank}$ of the first relevant result
- **NDCG (Normalized Discounted Cumulative Gain)**: Accounts for graded relevance and position

7.2 Generation Metrics

- **Faithfulness**: Does the response accurately reflect the retrieved context? (No hallucinated facts)
- **Answer Relevance**: Does the response actually answer the question?
- **Context Relevance**: Was the retrieved context useful for answering?

Frameworks like **RAGAS** provide automated metrics for these dimensions.

8 Advantages and Limitations

8.1 Advantages of RAG

1. **Up-to-Date Knowledge**: The knowledge base can be updated instantly without retraining.
2. **Reduced Hallucinations**: Grounding responses in retrieved facts significantly reduces fabrication.
3. **Source Attribution**: RAG can cite its sources, enabling verification and building trust.
4. **Domain Adaptation Without Training**: Add domain-specific documents without expensive fine-tuning.
5. **Cost Efficiency**: Much cheaper than training or fine-tuning large models.
6. **Data Privacy**: Sensitive data stays in your controlled infrastructure.

8.2 Limitations of RAG

→ query related information যদি knowledbase এ না থাকে।
→ hallucinated answer দিবে

1. **Retrieval Failures**: If retrieval fails to find relevant documents, the system fails silently or hallucinates anyway.

→ knowledge base টাই খারাপ

2. **Garbage In, Garbage Out**: Poor quality, outdated, or biased knowledge bases lead to poor outputs.
3. **Semantic Gap**: User queries often don't semantically match document content (questions vs. statements).
4. **Multi-Hop Reasoning**: Standard RAG struggles with questions requiring synthesis across multiple documents.
5. **Context Window Limits**: Can only inject limited context; very large document sets may not fit.

prompt → output (faster)
prompt → retrieval → output (slower)

6. **Latency**: The retrieval step adds latency to every query.
7. **Complex Engineering**: Production RAG requires managing embedding models, vector databases, chunking pipelines, and monitoring.

Advantages

- + Real-time knowledge updates
- + Reduced hallucinations
- + Source citations
- + No training required
- + Cost effective
- + Data stays private

Limitations

- × Silent retrieval failures
- × Knowledge base quality critical
- × Query-document semantic gap
- × Limited multi-hop reasoning
- × Added latency
- × Engineering complexity

Figure 7: Summary of RAG advantages and limitations.

9 When to Use RAG (and When Not To)

9.1 Use RAG When:

- **Knowledge changes frequently**: News, documentation, policies, regulations
- **You need factual accuracy with citations**: Legal, medical, financial applications
- **Working with proprietary data**: Internal documents, customer records, enterprise knowledge
- **Budget is limited**: RAG is much cheaper than fine-tuning
- **You need transparency**: Being able to show "this answer came from this source"

9.2 Consider Fine-Tuning Instead When:

- **You need to change model behavior**: Output format, style, tone, personality
- **Domain-specific reasoning is required**: Medical diagnosis, legal analysis
- **Knowledge is stable**: Information that doesn't change frequently
- **Speed is critical**: Fine-tuned models don't have retrieval latency

9.3 Use Both (Hybrid RAFT) When:

The most powerful approach often combines both: fine-tune for domain expertise and behavior, then use RAG for up-to-date facts. A fine-tuned medical model + RAG for latest research papers gives you both deep understanding and current information.

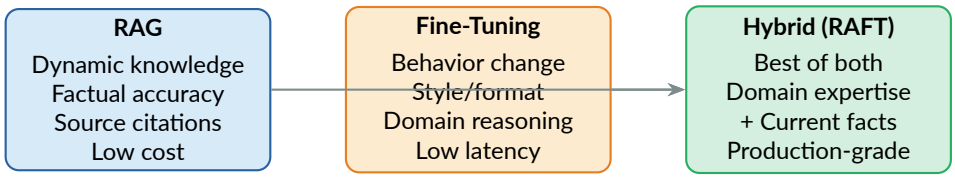


Figure 8: Decision framework: RAG for dynamic knowledge, fine-tuning for behavior, or combine both for optimal results.

10 Practical Implementation Considerations

10.1 Technology Stack

A typical RAG system requires:

- **Document Loaders**: LangChain, LlamaIndex, or custom parsers
- **Embedding Models**: OpenAI embeddings, Cohere, Sentence-Transformers, or open-source models
- **Vector Database**: Pinecone, Weaviate, Qdrant, Chroma, FAISS, or pgvector
- **Orchestration**: LangChain, LlamaIndex, or custom pipelines
- **LLM**: OpenAI GPT, Anthropic Claude, open-source models (Llama, Mistral)

10.2 Key Parameters to Tune

- **Chunk size**: 256–1024 tokens (experiment for your domain)
- **Chunk overlap**: 10–20% to preserve context across boundaries
- **Top-k**: Number of chunks to retrieve (typically 3–10)
- **Similarity threshold**: Minimum similarity score to include a result
- **Embedding model**: Match to your domain (general vs. specialized)

10.3 Common Failure Modes

1. **Missing Content**: The knowledge base doesn't contain relevant information
2. **Semantic Mismatch**: Query and document use different terminology
3. **Poor Chunking**: Relevant information split across chunks
4. **Context Overflow**: Too much retrieved context confuses the LLM
5. **Stale Data**: Knowledge base not updated regularly
6. **Hallucination Despite Context**: LLM ignores retrieved context

Summary

Let's recap what we've covered:

1. **The Problem**: LLMs have static knowledge, hallucinate, and can't access private data. RAG solves this by adding external retrieval.
2. **Architecture**: Three components—indexing (chunk, embed, store), retrieval (similarity search), and generation (augmented prompting).
3. **Mathematics**: Embeddings map text to vectors; cosine similarity or dot product measures semantic closeness; ANN algorithms enable fast search at scale.
4. **Chunking**: Strategy matters—fixed-size, semantic, recursive, or contextual chunking each have tradeoffs.

5. **Advanced Techniques**: Query rewriting, hybrid search, reranking, self-RAG, and GraphRAG extend basic capabilities.
6. **Evaluation**: Measure both retrieval quality (precision, recall) and generation quality (faithfulness, relevance).
7. **Tradeoffs**: RAG excels at dynamic, factual tasks but struggles with multi-hop reasoning and adds complexity.
8. **When to Use**: RAG for dynamic knowledge and citations; fine-tuning for behavior; hybrid for best results.

RAG represents a fundamental shift in how we build AI applications—from closed systems that only know their training data to open systems that can access and ground their responses in any knowledge source. As you build RAG systems, remember: the quality of your retrieval directly determines the quality of your generation. Invest in good chunking, robust retrieval, and comprehensive evaluation.